Robotic Arm for Automated Fiber Placement

1st Matthew Hutchinson
Mech&AE Dept.
UCLA
Los Angeles, CA
mahutchinson@ucla.edu

2nd Shirley Shah
Mech&AE Dept.
UCLA
Los Angeles, CA
sjshah@ucla.edu

3nd Jae-An Wang
Mech&AE Dept.
UCLA
Los Angeles, CA
jjwang319@engineering.ucla.edu

I. Overview

As composite manufacturing becomes more prominent in industry, the desire to automate the process increases as well. While composite materials are strong and lightweight, the manufacturing process can be quite cumbersome and requires intense and expert human operators. However, composites require a high level of precision. Factor such as fiber orientation, distance between fiber layers and air bubbles can greatly impact the quality of the composite piece. This leaves little room for error when manufacturing the pieces. Human operators are prone to causing error in the piece. Automatic Fiber Placement (AFP) is one such process of composite manufacturing. This process involves the placement of individual fiber tows in a specific orientation. Figure 1 depicts a schematic of the AFP setup. This process involves a compaction roller that lays the fiber band onto mold while applying pressure to eliminate air bubbles and ensure that the fiber is flush against the mold. The purpose of this project is to demonstrate the use of an impedance controller

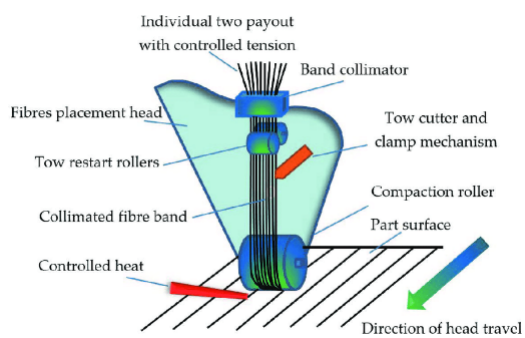

Fig. 1. Schematic of AFP Process with Compaction Error

A. Literature Review

The first inspiration for this project was the multi-robot cell presented in Malhan et al.. This paper proposes a solution for the hand-layup process, which is usually a tedious and multi-step task performed manually. This system consists of three arms, two for draping and one for both draping and rolling. The rolling arm is equipped with a cylindrical roller end effector. This cell also utilizes an impedance controller fed with a pre-planned trajectory [1].

Yan et al. discuss another composite manufacturing process that is automated - robotic fiber placement (AFP). This process consists of a single robot manipulator simultaneously heating, laying, and rolling aligned composite tows to form a single ply in the layup. This paper focused exclusively on trajectory generation for full roller head coverage of a curved mold. The AFP control problem, which is simpler than the cooperative setup from [2] since it only requires a single robotic arm to perform, is left for us to solve.

II. Design Objectives

The main project objectives were to design a controller for the AFP task as described by Yan et al. and create a simulation to test the performance of the controller.

The controller design objectives were the following, assuming a given preplanned trajectory appropriate for the task:

1) Measured along or about each world frame axis, the end effector frame stays within 2.5mm of the desired position and $0.01\,\mathrm{deg}$ of the desired orientation (measured using roll, pitch, and yaw). This is meant to ensure full roller coverage of the mold, assuming an ideal trajectory.
2) The applied force normal to the mold surface remains at $50 \pm 10N$ to provide a relatively constant rolling force.
3) The applied force along each non-constrained direction is under 10N, and the applied torque about the local normal to the mold is under 0.1Nm, so that excessive deformation of the composite ply does not occur.
4) The applied torque about each axis of the mold's local tangent plane is under 1Nm. These are relatively relaxed constraints since these are less likely to significantly impact the product quality.
5) The above objectives are achieved using a 6DOF manipulator, which allows for control of position and orientation in all axes relative to the curved mold.
6) The above objectives are achieved using an impedance controller in order to demonstrate and expand on concepts learned from lecture.

Here we note that although this project originally intended to use a curved parametric surface for the mold shape, due to time constraints, we simplified the problem by using a flat rectangular mold instead. Increasing the problem complexity by switching to a parametric surface mold is one of our future work items in Section VI: Future Work.

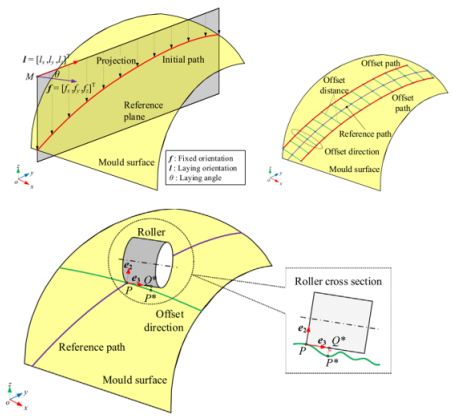

Fig. 2. Path generation method described by Yan et al.

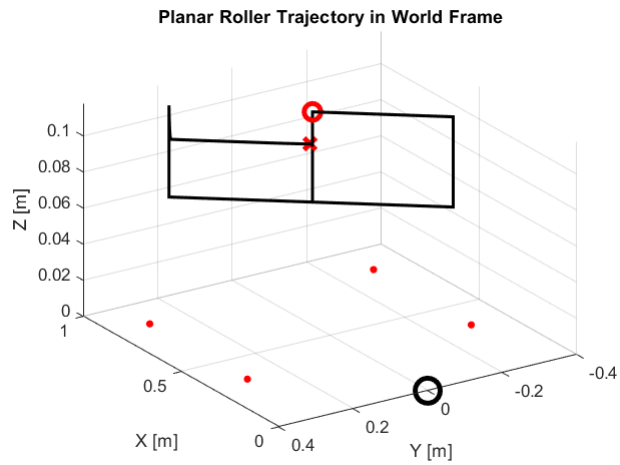

Fig. 3. Trajectory segment for a single path out of a full set.

A. Controller Assumptions and Evaluation

For this simulations, the following assumptions were made:

1) The forces acting on the end effector are not negligible. Contact forces are assumed to be caused by normal contact and friction, and the roller experiences some arbitrary damping.
2) The geometry of the mold is already known and can be parameterized as $[x, y, z] = S(u, v)$, which leads to a pre-planned trajectory.
3) There is perfect force and orientation feedback.
4) Due to the planar nature of the model, the desired angular velocities of the end effector are zero
5) Due to the planar nature of the model, the desired end effector orientation is constant

III. Methods

The following section describes the steps taken to simulate the AFP problem.

A. Trajectory Generation

The methods described by Yan et al. were used for trajectory generation. To summarize, for a given mold surface parameterized as $[x, y, z] = S(u, v)$ with $u, v \in [0, 1]$, an initial reference path is generated from the intersection between the surface and a reference plane aligned with the desired composite tow laying direction. More paths are iteratively generated from this initial reference path using more surface-plane intersections; these subsequent planes are offset from their respective reference planes along the their normal axis by a maximum allowable distance which is dependent on the local curvature of the mold surface, as shown in Fig. 2. This is repeated until the set of paths provide full coverage of the surface.

These methods were described analytically in the reference, but were implemented using numerical approximation; the code used is included with this paper. Additionally, the decision to use a planar mold simplifies this problem to be trivial, as a constant offset between parallel linear paths may be used due to the lack of surface curvature.

The set of paths was extended into a full trajectory by first discretizing each path into waypoints, and then adding waypoints to connect the ends of each path. Trajectory points in between each waypoint were defined using a trapezoidal velocity profile with a set max velocity and zero-velocity endpoints for each phase of the path.

For a planar mold, two surface waypoints were sufficient for each path, and each path is divided into several phases for the roller (Fig. 3):

1) Descend vertically to the surface of the mold at the path midpoint
2) Try to descend through the surface of the mold to the desired waypoint
3) Roll from the path midpoint to one endpoint
4) Rise vertically to a set offset distance above the mold
5) Move horizontally back above the path midpoint
6) Descend vertically to the surface of the mold at the path midpoint again
7) Try to descend through the surface of the mold to the desired waypoint again
8) Roll from the path midpoint to the other endpoint
9) Rise vertically to the offset height
10) Move to the offset height over the next path midpoint or the home position if finished

With this, the reference trajectory is known with respect to the mold; it is transformed into the world frame coordinates later once the transform from mold frame to the manipulator base frame is known. An example for the planar mold $45\,\mathrm{deg}$ laying angle case is shown in Fig. 4

B. Mesh Generation

A mold 3D model is needed for the simulation environment for both visual purposes and for contact modeling in Drake, which will be discussed in the next section. Since only the parametric equation $S(u, v)$ for the mold surface is known, there are several steps to create a workable model. The steps

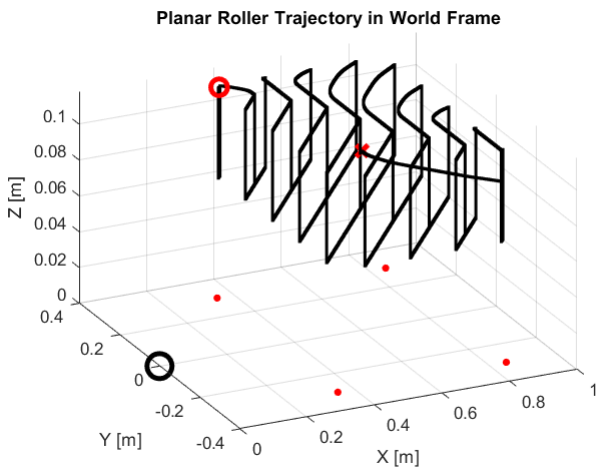

Fig. 4. Full roller trajectory for the $45\,\mathrm{deg}$ case shown in the base frame.

are briefly described here, but the details are embedded in the attached code.

First, in order to create a point cloud representing the mold, a dense grid of (u, v) values is used to generate a large set of surface points. An example of this is seen in Fig 5. The bottom of the mold is generated by projecting these points to the XY plane; the surface points may be displaced vertically to give the desired mold height. The sides of the mold are generated by interpolating the boundaries of the mold surface and bottom. Approximate surface normals at each point are calculated numerically. The point locations and normals are then saved to a text file.

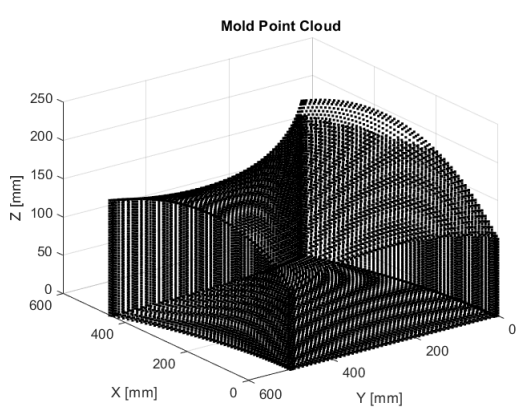

Fig. 5. Point Cloud Representation of Mold

Open3D, a 3D data processing library, is used to convert the point cloud data to a triangle mesh using the Ball Pivoting Algorithm, as described in [5]. This mesh can then be repaired and altered through any CAD or meshing software and exported as an STL.

The STL file can be converted to an OBJ triangle mesh using Meshio in Python, which is used for the visual aspect of the mold in the simulation. The collision model is generated by first using fTetWild, a free open-source meshing software, to convert the triangular mesh to a tetrahedral mesh. This is then converted to a VTK file, again using Meshio. This allows the potentially nonconvex mold to be used in Drake's hydroelastic contact model in the simulation. An example of the final product can be seen in Fig 6

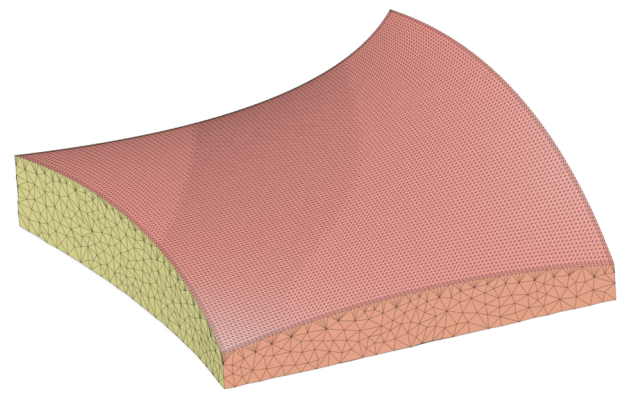

Fig. 6. Mesh Generated from Surface Points

C. Simulation Setup

The simulation code was written in Python, specifically utilizing PyDrake. Pydrake is a Python wrapper for the Drake C++ toolbox, which is useful for modelling and simulation. To simulate a force sensor, PyDrake's in built contact model was used to "measure" the force at the end effector. The robot arm chosen was the Universal Robot 5e (UR5e). The URDF file was found in a public GitHub database [3].

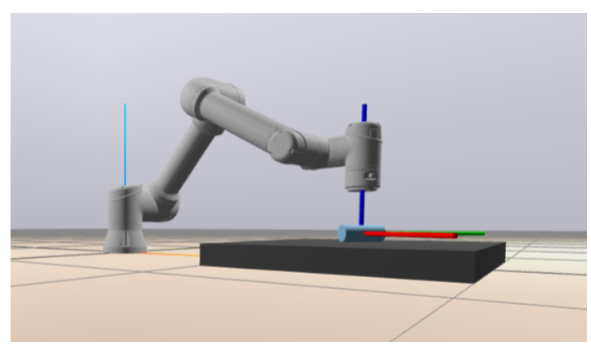

Fig. 7. Simulation setup of UR5E manipulator rolling a planar mold.

D. Impedance Controller

1) Derivation: An impedance controller was implemented in order to control the force at the end effector [**?**]. Since the roller is also required to follow a trajectory, the controller must be implemented in operational task space. Assuming that the arm is equipped with a wrist sensors and receives orientation feedback in the operational space, the new error definition can be set to Eq 1 and Eq 2. Eq. 3 can be used to calculate the control output. To get the desired accelerations, Eq 3 can be

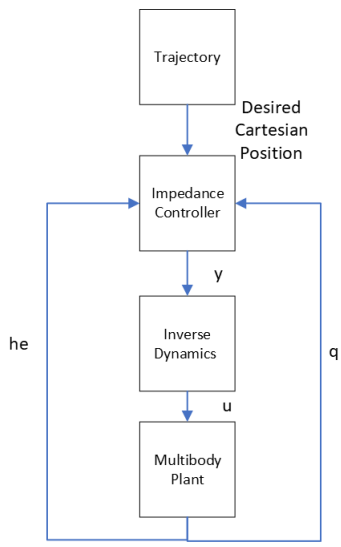

Fig. 8. High Level Impedance Control Diagram

used. This output is then input into inverse dynamics to get the desired torques for the robotic arm. The high level diagram for the controller implementation is seen in Fig 8

$$\tilde{x} = \begin{bmatrix} o^d_{d,e} \\ \phi_{d,e} \end{bmatrix} \quad (1) \qquad \dot{\tilde{x}} = \begin{bmatrix} \dot{o}^d_{d,e} \\ \dot{\phi}_{d,e} \end{bmatrix} \quad (2)$$

$$y = J^{-1}_{Ad} M^{-1}_d [K_d \dot{\tilde{x}} + K_p \tilde{x} - M_d \dot{J}_{Ad} + M_d \dot{b} - h^d_e] \quad (3)$$

$$b = \begin{bmatrix} R^T_d \dot{(o)}_d + S(w^d_d) o^d_{d,e} \\ T^{-}1(\phi_{d,e}) w^d_d \end{bmatrix} \quad (4)$$

$$\dot{\tilde{x}} = -J_{Ad}\dot{q} + b \quad (5)$$

E. K_D calculation

The value for the K_D gains were calculated by assuming the unconstrained directions to be x and y and the constrained direction to be z, roll, pitch and yaw. For the unconstrained direction, the k_d was solved using Eq. 6 and assuming ζ to be 1. For the constrained direction, Eq. 7 was used under the same assumption for ζ. The variable k is the stiffness of the environment. These were calculated from the hydro-elastic model of the mold. For both cases, $k = 10^6 N/m$

$$\zeta_{uc} = \frac{k_D}{2\sqrt{m_d k_P}} \quad (6)$$

$$\zeta_c = \frac{k_D}{2\sqrt{m_d(k_P + k}} \quad (7)$$

IV. Results

To tune the gains, only a single pass within the trajectory was considered. This was to save computation time when testing various values for K_P and M_D. Various values were tested to try and meet the criteria stated above. Once a set of gains that meet the criteria was found, two other set of gains was tried: one set that was lower than the original gain, and one set that was higher that the original gain. These three gain cases are seen in Table I. The simulation result from these values can be seen in Fig 9, Fig 10, Fig 11. As expected, Case 1 meets the requirements but deviations from this case causes the system to fail. When applying the gains used in Case 1 to the full trajectory, it can be seen from Fig 12, Fig 13 and Fig 14 that the controller does fail at some points in the trajectory. Especially in the force, there are some undesirable spikes in the Z direction, indicating the roller is exerting a non-constant force on the mold, which in practical application,lead to deformity. With more time and more testing, alternative gain values can be found to eliminate this problem.

TABLE I
GAIN VALUES USED FOR CASES

	Case 1
M_D	$diag(1\times10^2, 1\times10^2, 2\times10^6, 1\times10^4, 1\times10^3, 1\times10^4)$
K_P	$diag(1\times10^6, 1\times10^5, 1\times10^{10}, 1\times10^7, 1\times10^7, 1\times10^9)$
	Case 2
M_D	$diag(1\times10^3, 2\times10^2, 3\times10^6, 2\times10^4, 2\times10^3, 2\times10^4)$
K_P	$diag(2\times10^6, 2\times10^5, 2\times10^{10}, 2\times10^7, 2\times10^7, 2\times10^9)$
	Case 3
M_D	$diag(2\times10^1, 5\times10^1, 2\times10^2, 1\times10^2, 1\times10^2, 1\times10^3)$
K_P	$diag(1\times10^3, 1\times10^2, 1\times10^3, 1\times10^3, 1\times10^3, 1\times10^4)$

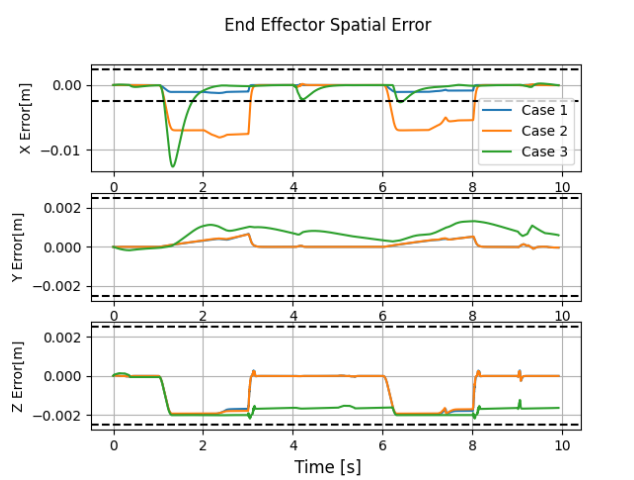

Fig. 9. Spatial Error For Three Different Test Cases: Single Pass

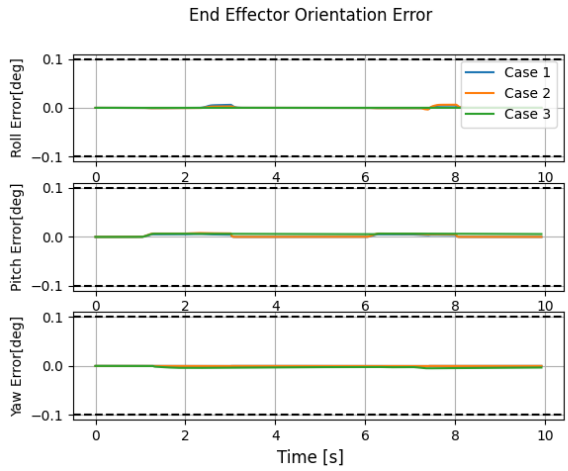

Fig. 10. Orientation Error For Three Different Test Cases: Single Pass

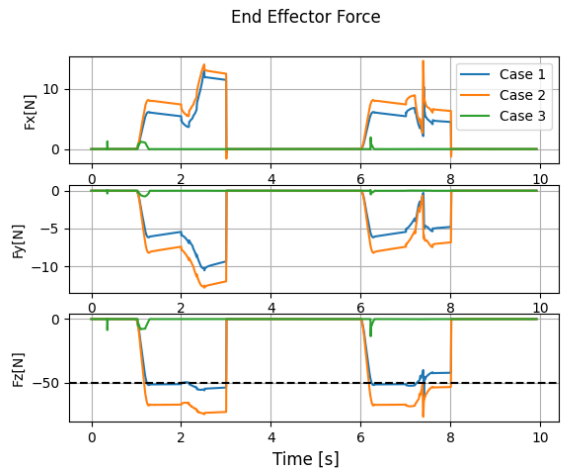

Fig. 11. Force For Three Different Test Cases: Single Pass

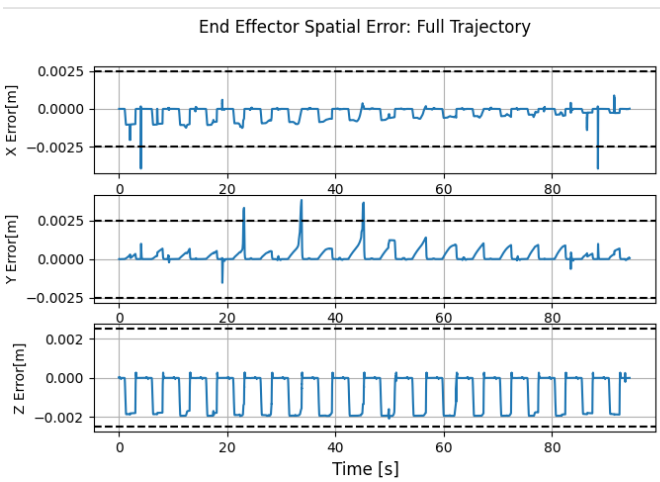

Fig. 12. Spatial Error at Full Trajectory

V. Discussion

All in all, this project was an educational experienece, especially in working with Pydrake. During gain tuning, the

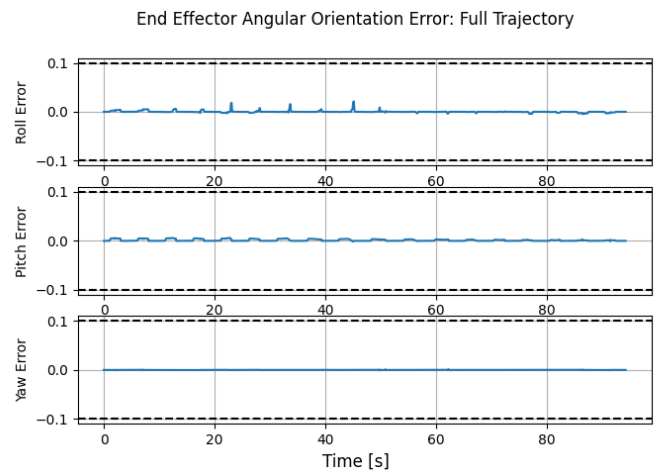

Fig. 13. Orientation Error at Full Trajectory

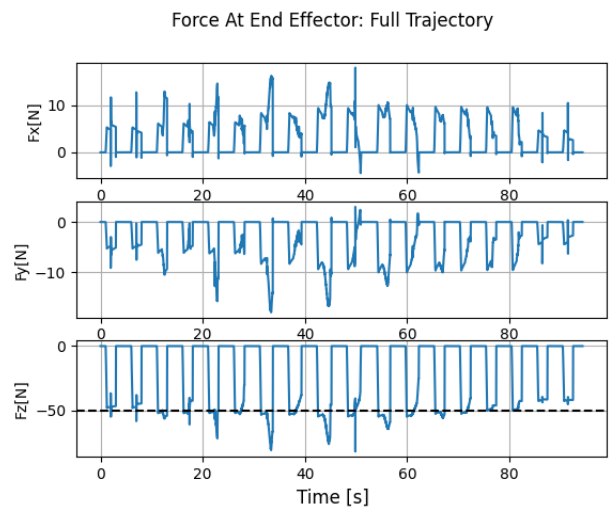

Fig. 14. Force at Full Trajectory

team saw that the gain on Z must be increased quite a big in order to meet the criteria. This is most likely due to the Z direction being the constrained direction. This controller is only limited to the planar case, as it assumes zero angular velocities. In terms of technical limitations, PyDrake required a huge learning curve in order to implement. There were many roadblocks before a working simulation was obtained. There are also uncertainties regarding PyDrakes contact modelling system. With these uncertainties and roadblocks, more time than expected was used in debugging as opposed to running actual experiments.

VI. Future Work

For future works, a major change would be to implement a non-planar mold. This would affect the end effector orientation making the assumption of zero angular velocities and constant end effector orientation inapplicable. It would also be interesting to see how this controller will compare to other controllers, such as a hybrid force-motion controller. If this

project were to be continued, the contact model built into needs to be validated to ensure that the fprces readings are accurate to this application.

VII. Acknowledgment

We would like to acknowledge Professor Veronica Santos and TA Cole Ten for the feedback and guidance for this project.

VIII. References

References

[1] Malhan, R. K., Shembekar, A. V., Kabir, A. M., Bhatt, P. M., Shah, B., Zanio, S., Nutt, S., & Gupta, S. K. (2021). Automated planning for robotic layup of composite prepreg. Robotics and Computer-integrated Manufacturing, 67, 102020. https://doi.org/10.1016/j.rcim.2020.102020

[2] Yan, L., Chen, Z. C., Shi, Y., Mo, R. (2014). An accurate approach to roller path generation for robotic fibre placement of free-form surface composites. Robotics and Computer-integrated Manufacturing, 30(3), 277–286. https://doi.org/10.1016/j.rcim.2013.10.007

[3] Daniella. (n.d.). GitHub - Daniella1/urdf_files_dataset. GitHub. https://github.com/Daniella1/urdf_files_dataset

[4] Siciliano, B., Sciavicco, L., Villani, L., Oriolo, G. (2010). Robotics: Modelling, Planning and Control. Springer Science Business Media.

[5] Poux, F., PhD. (2024, January 24). Generate 3D meshes from point clouds with Python — Towards Data Science. Medium. https://towardsdatascience.com/5-step-guide-to-generate-3d-meshes-from-point-clouds-with-python-36bad397d8ba

IX. Appendix: Team Member Contribution

TABLE II
Team Member Contribution

Team Member	Contribution
Jae-An Wang	Trajectory and Mold Generation, Controller Development, Report Writing
Shirley Shah	Controller Derivation and Development, Report Writing, Integrating Code
Matthew Hutchinson	Report Writing, Controller Development

X. Appendix: Main Python Code

```python
from __future__ import annotations
import faulthandler
import pandas as pd
from projectHelperFunctions import TruncateVector
import math
from dataclasses import dataclass
from functools import cached_property
import matplotlib.pyplot as plt
import pydrake.multibody.parsing
from numpy.typing import NDArray
from pydrake.multibody.parsing import AddFrame
import scipy as sp
from pydrake.systems.controllers import InverseDynamics
from scipy.spatial.transform import Rotation as R
from pydrake.multibody.plant import ContactResults
from pydrake.common.value import Value
from pydrake.geometry import Meshcat, SceneGraph, MeshcatVisualizer
from pydrake.math import RigidTransform, RotationMatrix, RollPitchYaw
from pydrake.multibody.parsing import Parser
from pydrake.multibody.inverse_kinematics import InverseKinematics
from pydrake.multibody.plant import (
    AddMultibodyPlantSceneGraph, ContactModel,
    MultibodyPlant_, MultibodyPlant, MultibodyPlantConfig, AddMultibodyPlant)
from pydrake.multibody.tree import (
    MultibodyForces,
    FixedOffsetFrame,
    JacobianWrtVariable,
    RevoluteJoint
)
from pydrake.multibody.math import SpatialForce
from pydrake.visualization import AddFrameTriadIllustration
from pydrake.systems.analysis import Simulator
from numpy import cos, sin
from pydrake.systems.framework import (
    DiagramBuilder,
    Diagram,
    LeafSystem,
    Context,
    DiscreteValues,
    BasicVector,
)
from pydrake.systems.primitives import ConstantValueSource, MatrixGain, LogVectorOutput
from mechae263C_helpers.drake import LinearCombination
import matplotlib.pyplot as plt
import numpy as np
from numpy.typing import NDArray
from pydrake.systems.analysis import Simulator
from pydrake.systems.framework import (
    Context,
    Diagram,
    DiagramBuilder,
    InputPort,
    OutputPort,
)
```

```python
from pydrake.systems.primitives import (
    MatrixGain,
    PassThrough,
    ZeroOrderHold,
    LogVectorOutput,
)

from mechae263C_helpers.plotting import plot_diagram
from mechae263C_helpers.plotting import plot_diagram
# from ImpedanceLeafSystem import OperationalImpedanceController
# from ForwardKinsLeafSystem import forwardkin
from projectHelperFunctions import moveMold, get_q_initial, vis_frame, TruncateVector

# from trajectory_mold import PrePlannedTrajectoryMold


MESHCAT_PORT = 7000


class PrePlannedTrajectoryMold(LeafSystem):
    def __init__(
            self,
            name: str,
            times: NDArray[np.double],
            od: NDArray[np.double],
            odd: NDArray[np.double],
            oddd: NDArray[np.double]
    ):
        super().__init__()
        self.times = np.asarray(times, dtype=np.double)
        self.od = np.asarray(od, dtype=np.double)
        self.odd = np.asarray(odd, dtype=np.double)
        self.oddd = np.asarray(oddd, dtype=np.double)

        self.ix: int = 0
        self.DeclareVectorOutputPort(
            name=name, size=9, calc=self.calc_output
        )

    def calc_output(self, context: Context, output: BasicVector):
        t = context.get_time()

        if t >= self.times[self.ix]:
            self.ix += 1

        self.ix = min(self.ix, len(self.times) - 1)
        od_curr = self.od[:, self.ix]
        odd_curr = self.odd[:, self.ix]
        oddd_curr = self.oddd[:, self.ix]
        traj_vector = np.hstack((od_curr, odd_curr, oddd_curr))
        #print("hello: ", traj_vector)
        #output.set_value(self.values[:, self.ix])  # noqaq
        #print("t: ", t)
        output.set_value(traj_vector)
```

```python
class WrenchOutput(LeafSystem):
    def __init__(self):
        super().__init__()  # Don't forget to initialize the base class.
        self.DeclareAbstractInputPort(
            name="wrenchInput",
            model_value=Value(ContactResults())
        )
        self.DeclareVectorOutputPort(
            name="wrenchOutput",
            size=6,
            calc=self.update_spatial_force
        )

    def update_spatial_force(self, context: Context, output: BasicVector):
        contact_results = self.get_input_port().Eval(context)
        if contact_results.num_hydroelastic_contacts() == 0:
            zero_force = np.array([0, 0, 0, 0, 0, 0], np.float64)
            output.set_value(zero_force)
        else:
            if contact_results.num_hydroelastic_contacts() > 1:
                print("multiple contacts!!!")
            contact_info = contact_results.hydroelastic_contact_info(0)  # assume first contac
            spatial_force = contact_info.F_Ac_W()
            f = spatial_force.translational()
            tau = spatial_force.rotational()
            print(spatial_force)
            spatial_force_arr = np.concatenate((f, tau), axis=None) / 1000000
            output.set_value(spatial_force_arr)


class OperationalImpedanceController(LeafSystem):
    def __init__(self,
                 M_d: NDArray[np.double],
                 K_P: NDArray[np.double],
                 K_D: NDArray[np.double],
                 plant: MultibodyPlant,
                 R_0D,  # assuming planar model
                 w_0D  # assuming planar model
                 ):
        super().__init__()
        self.plant = plant
        self.M_d = M_d
        self.K_P = K_P
        self.K_D = K_D
        self.R_0D = R_0D
        self.w_0D = w_0D

        # Variables for logging - jank
        self.x_tilde = np.zeros((6, 1))
        self.x_tilde_dot = np.zeros((6, 1))
        self.h_e = np.zeros((6,1))

        self.DeclareVectorInputPort(name="states",
                                    size=14  # 7 DOF including roller Joint
                                    )
        self.DeclareVectorInputPort(name="FK_Output",
```

```
                                size=12  # o_e, rpy_e, o_e_dot, w_e
                                )
        self.DeclareVectorInputPort(name="Desired Translational State",
                                size=9
                                )
        self.DeclareVectorInputPort(name="tau_h_e",  # THIS IS ACTUALLY GETTING GENERALIZED CO
                                size=6
                                )
        # Main Output Port
        self.DeclareVectorOutputPort(name="y",
                                 size=7,
                                 calc=self.calc_y
                                 )
        # Logging Ports
        self.DeclareVectorOutputPort(name="x_tilde",
                                 size=6,
                                 calc=self.get_x_tilde
                                 )
        self.DeclareVectorOutputPort(name="x_tilde_dot",
                                 size=6,
                                 calc=self.get_x_tilde_dot
                                 )
        self.DeclareVectorOutputPort(name="h_e",
                                 size=6,
                                 calc=self.get_h_e

        )

    def calc_TA(self, rpy):
        # Calculate mapping from rpy velocities to tool frame angular velocity
        T_rpy2w = np.array([[np.cos(rpy[2]) * np.cos(rpy[1]), -np.sin(rpy[2]), 0],
                            [np.sin(rpy[2]) * np.cos(rpy[1]), np.cos(rpy[2]), 0],
                            [-np.sin(rpy[1]), 0, 1]])
        T_A = np.vstack((np.hstack((np.eye(3), np.zeros((3, 3)))),
                         np.hstack((np.zeros((3, 3)), T_rpy2w))))
        if np.linalg.cond(T_A) > 1e8:
            print("WARNING: T_A is ill-conditioned! There may be numerical errors present")

        return T_A

    def calc_des_analytical_Jacobian(self, plant_context, rpy_0E, w_0E):
        # Calculate current pose error from FK feedback, desired pose
        R_0E = RotationMatrix(RollPitchYaw(rpy_0E)).matrix()
        R_DE = RotationMatrix(self.R_0D.T @ R_0E)  # Calculate rotational error matrix
        rpy_DE = np.asarray(R_DE.ToRollPitchYaw().vector())
        TA_DE = self.calc_TA(rpy_DE)

        # Calculate geometric Jacobian; note that angular velocities are above translational v
        base_frame = self.plant.GetFrameByName("base_link")
        tool_frame = self.plant.GetFrameByName("roller_attachment_parent")
        J_g_v = self.plant.CalcJacobianTranslationalVelocity(
            context=plant_context,
            with_respect_to=JacobianWrtVariable.kQDot,
            frame_B=tool_frame,
            p_BoBi_B=np.array([0, 0, 0]),
            frame_A=base_frame,  # try world frame??
```

```
        frame_E=base_frame,
    )[:, :-1]  # last column is from additional roller joint
    J_g_w = self.plant.CalcJacobianAngularVelocity(
        context=plant_context,
        with_respect_to=JacobianWrtVariable.kQDot,
        frame_B=tool_frame,
        frame_A=base_frame,  # try world frame??
        frame_E=base_frame,
    )[:, :-1]  # last column is from additional roller joint
    J_G = np.vstack((J_g_v, J_g_w))

    # Calculate analytical Jacobian to desired frame
    R_0d_block = np.block([[self.R_0D.T, np.zeros((3, 3))],
                           [np.zeros((3, 3)), self.R_0D.T]])
    JA_D = np.linalg.inv(TA_DE) @ R_0d_block @ J_G
    return JA_D, J_G, rpy_DE

def get_S(self, w):
    # Returns skew symmetric form of input vector w.
    wx = w[0]
    wy = w[1]
    wz = w[2]
    S = np.array([[0, -wz, wy], [wz, 0, -wx], [-wy, wx, 0]])
    return S

def get_Tinvdot(self, theta, psi, theta_dot, psi_dot):
    # Gimbal lock warning - avoid pitch close to +- pi/2
    if abs(abs(theta) - np.pi / 2) <= 1e-8:
        print("WARNING: ImpedanceLeafSystem: get_Tinvdot: Pitch angle is near +-90deg! "
              "There may be numerical errors present due to gimbal lock!")

    # T_inverse expression obtained from MATLAB symbolic calc; derivative done elementwise
    TiD = np.zeros(shape=(3, 3), dtype=np.double)
    TiD[0, 0] = (-np.cos(theta) * np.sin(psi) * psi_dot + np.sin(theta) * np.cos(psi) * the
            np.cos(theta) ** 2)
    TiD[1, 0] = -np.cos(psi) * psi_dot
    TiD[2, 0] = np.cos(psi) * theta_dot - np.sin(theta) * np.cos(theta) * np.sin(psi) * psi
    TiD[0, 1] = (np.cos(theta) * np.cos(psi) * psi_dot + np.sin(psi) * np.sin(theta) * the
            np.cos(theta) ** 2)
    TiD[1, 1] = -np.sin(psi) * psi_dot
    TiD[2, 1] = (np.sin(psi) * theta_dot + np.sin(theta) * np.cos(theta) * np.cos(psi) * p
            np.cos(theta) ** 2)
    return TiD

def calc_des_analytical_Jacobian_dot_q_dot(self, plant_context, rpy_0E, q_dot, w_0E, w_0D=
    # Calculate Jacobians and angular rates
    JA_D, J_G, rpy_DE = self.calc_des_analytical_Jacobian(plant_context, rpy_0E, w_0E)
    TA_DE = self.calc_TA(rpy_DE)
    w_0_DE = w_0E - w_0D
    rpy_DE_dot = np.linalg.inv(TA_DE)[3:, 3:] @ self.R_0D.T @ w_0_DE
    T_DE_inv_dot = self.get_Tinvdot(rpy_DE[1], rpy_DE[2], rpy_DE_dot[1], rpy_DE_dot[2])
    TA_DE_inv_dot = np.vstack((np.zeros((3, 6)),
                               np.hstack((np.zeros((3, 3)), T_DE_inv_dot))))  # THIS MAY B

    # Calculate bias spatial acceleration (J_dot * q_dot)
    base_frame = self.plant.GetFrameByName("base_link")
```

```
        tool_frame = self.plant.GetFrameByName("roller_attachment_parent")
        J_dot_q_dot = self.plant.CalcBiasSpatialAcceleration(
            context=plant_context,
            with_respect_to=JacobianWrtVariable.kV,  # v and qdot are the  same
            p_BoBp_B=np.array([0, 0, 0]),
            frame_B=tool_frame,
            frame_A=base_frame,
            frame_E=base_frame,
        )
        J_dot_q_dot = np.hstack([J_dot_q_dot.translational(), J_dot_q_dot.rotational()])

        w_D_0D = self.R_0D.T @ w_0D
        R_D0_dot = self.get_S(w_D_0D) @ self.R_0D.T
        R_D0_dot_block = np.block([[R_D0_dot, np.zeros((3, 3))],
                                   [np.zeros((3, 3)), R_D0_dot]])
        R_D0_block = np.block([[self.R_0D.T, np.zeros((3, 3))],
                               [np.zeros((3, 3)), self.R_0D.T]])
        # print("TA_DE : ", TA_DE)
        # print("R_D0 : ", R_D0_block)
        # print("Jdqd : ", J_dot_q_dot)
        # print("R_D0_dot : ", R_D0_dot_block)
        # print("TA_DE_inv_dot : ", TA_DE_inv_dot)
        # print("J_G: ", J_G)
        # print("q_dot : ", q_dot)
        JA_D_dot_q_dot = np.linalg.inv(TA_DE) @ R_D0_block @ J_dot_q_dot \
                         + np.linalg.inv(TA_DE) @ R_D0_dot_block @ J_G @ q_dot \
                         + TA_DE_inv_dot @ R_D0_block @ J_G @ q_dot

        return JA_D_dot_q_dot, JA_D, J_G, rpy_DE, rpy_DE_dot

    def getB(self, w_0D, od_0_0D, od_D_DE, T_DE_inv):
        top = self.R_0D.T @ od_0_0D + self.get_S(self.R_0D.T @ w_0D) @ od_D_DE
        bottom = T_DE_inv @ self.R_0D.T @ w_0D
        B = np.hstack((top, bottom))
        return B

    def getBdotPlanar(self, odd_0D):
        top = self.R_0D.T @ odd_0D
        bottom = np.zeros(3)
        b_dot = np.hstack((top, bottom))
        return b_dot

    def calc_y(self, context: Context, output: BasicVector):
        # Get current sim info from context
        state = self.get_input_port(0).Eval(context)
        FK_Res = self.get_input_port(1).Eval(context)
        Des_trans = self.get_input_port(2).Eval(context)
        tau_h_e = self.get_input_port(3).Eval(context)
        plant_context = self.plant.GetMyContextFromRoot(root_context)

        # Unpack desired translational state
        o_0_0D = Des_trans[:3]
        od_0_0D = Des_trans[3:6]
        odd_0_0D = Des_trans[6:]

        # Unpack forward kinematics
```

```
        o_0_0E = FK_Res[:3]
        rpy_0E = FK_Res[3:6]
        od_0_0E = FK_Res[6:9]
        w_0_0E = FK_Res[9:]

        # Get intermediate matrices
        q_dot = state[7:-1]  # Exclude roller joint states
        JA_D_dot_q_dot, JA_D, J_G, rpy_DE, rpy_DE_dot = (
            self.calc_des_analytical_Jacobian_dot_q_dot(plant_context, rpy_0E, q_dot, w_0_0E, 
        JA_D_inv = np.linalg.inv(JA_D)
        M_d_inv = np.linalg.inv(self.M_d)
        T_DE_inv = np.linalg.inv(self.calc_TA(rpy_DE)[3:, 3:])

        # Calculate actual h_e
        self.h_e = -np.linalg.inv(J_G.T) @ tau_h_e  # Check sign

        # Calculate rotation matrices
        R_0E = RotationMatrix(RollPitchYaw(rpy_0E))
        R_DE = RotationMatrix(RollPitchYaw(rpy_DE))
        R_D0_block = np.block([[self.R_0D.T, np.zeros((3, 3))],
                               [np.zeros((3, 3)), self.R_0D.T]])

        # Calculate errors
        o_0_DE = o_0_0E - o_0_0D
        # rpy_DE = rpy_DE
        od_0_DE = od_0_0E - od_0_0D
        w_0_DE = w_0_0E - self.w_0D
        o_D_DE = self.R_0D.T @ o_0_DE
        h_e_D = R_D0_block @ self.h_e

        self.x_tilde = -np.hstack([o_D_DE, rpy_DE])
        #print(x_tilde)

        # Calculate x_tilde_dot
        w_D_0D = self.R_0D.T @ self.w_0D
        od_D_DE = self.R_0D.T @ od_0_DE - self.get_S(w_D_0D) @ self.R_0D.T @ o_0_DE
        self.x_tilde_dot = -JA_D @ q_dot + self.getB(self.w_0D, od_0_0D, od_D_DE, T_DE_inv)

        inner_y = self.K_D @ self.x_tilde_dot + self.K_P @ self.x_tilde - self.M_d @ JA_D_dot_
            odd_0_0D) - h_e_D
        y = JA_D_inv @ M_d_inv @ inner_y
        y_ext = np.hstack((y, 0))  # extra term shouldn't matter?
        #print("controller inner_y: ", inner_y)
        output.set_value(y_ext)
        # return y

    def get_x_tilde(self, context: Context, output: BasicVector):
        output.set_value(self.x_tilde)

    def get_x_tilde_dot(self, context: Context, output: BasicVector):
        output.set_value(self.x_tilde_dot)

    def get_h_e(self, context: Context, output: BasicVector):
        output.set_value(self.h_e)
```

```python
class forwardkin(LeafSystem):
    # Taking the context of the class, will return a 12 by 1 vector of end effect position,ori
    # transational velocity and angular velocity
    def __init__(self, plant: MultibodyPlant):
        super().__init__()
        self.plant = plant

        self.DeclareVectorInputPort("arm_state", size=14)
        self.DeclareVectorOutputPort("Fk_Output", size=12, calc=self.setOutput)

    def fk(self, plant_context):
        # Evaluate the end effector position
        base_frame = self.plant.GetFrameByName("base_link")
        tool_frame = self.plant.GetFrameByName("roller_attachment_parent")
        T_0E = self.plant.CalcRelativeTransform(
            context=plant_context,
            frame_A=base_frame,
            frame_B=tool_frame,
        )
        x_e_full = np.array([T_0E.translation(), RollPitchYaw(T_0E.rotation()).vector()])
        x_e = x_e_full.flatten()
        return x_e

    def Jacobian(self, plant_context, q_dot):
        base_frame = self.plant.GetFrameByName("base_link")
        tool_frame = self.plant.GetFrameByName("roller_attachment_parent")
        J_g_v = self.plant.CalcJacobianTranslationalVelocity(
            context=plant_context,
            with_respect_to=JacobianWrtVariable.kQDot,
            frame_B=tool_frame,
            p_BoBi_B=np.array([0, 0, 0]),
            frame_A=base_frame,  # try world frame??
            frame_E=base_frame,
        )[:, :-1]  # last column is from additional roller joint
        J_g_w = self.plant.CalcJacobianAngularVelocity(
            context=plant_context,
            with_respect_to=JacobianWrtVariable.kQDot,
            frame_B=tool_frame,
            frame_A=base_frame,  # try world frame??
            frame_E=base_frame,
        )[:, :-1]
        J_G = np.vstack((J_g_v, J_g_w))
        x_e_dot = J_G @ q_dot
        return x_e_dot

    def setOutput(self, context, output):
        # Get current state
        plant_context = self.plant.GetMyContextFromRoot(root_context)
        state = self.get_input_port().Eval(context)
        q = state[:6]
        q_dot = state[7:-1]

        # Output end effector position and velocity
        x_e = self.fk(plant_context)
        #print("This is x_e",x_e)
        x_e_dot = self.Jacobian(plant_context, q_dot)
```

```
        #print("This is x_e_dot",x_e_dot)
        x_e_list = x_e.tolist()

        x_e_dot_list = x_e_dot.tolist()
        x_e_list.extend(x_e_dot_list)
        #print(x_e_list)
        output_vec = np.array(x_e_list)
        #print("This is output vec",output_vec)
        output.set_value(output_vec)


def get_input_trajectory(csv_filepath):
    # Read CSV file with trajectory
    traj = pd.read_csv(csv_filepath)
    traj_times = np.array(traj["Times"].tolist())
    odx = np.array(traj["odx"].tolist())
    ody = np.array(traj["ody"].tolist())
    odz = np.array(traj["odz"].tolist())

    oddx = np.array(traj["oddx"].tolist())
    oddy = np.array(traj["oddy"].tolist())
    oddz = np.array(traj["oddz"].tolist())

    odddx = np.array(traj["odddx"].tolist())
    odddy = np.array(traj["odddy"].tolist())
    odddz = np.array(traj["odddz"].tolist())

    od = np.vstack((odx, ody, odz))
    odd = np.vstack((oddx, oddy, oddz))
    oddd = np.vstack((odddx, odddy, odddz))

    return traj_times, od, odd, oddd


def buildDiagram(
        plant: MultibodyPlant,  # Multibody Plant
        builder: DiagramBuilder,
        models,  # list of bodies within Scene Graph

        M_d: NDArray,  #6 by 6 Matrix
        K_P: NDArray,  #6 by 6 Matrix
        K_D: NDArray,  #6 by 6 Matrix

        R_0D: NDArray[np.double],  #3 by 3 Desired Rotation Matrix
        w_0D: NDArray[np.double],  #3 by 1 Desired omega
        # p_0E_initial, #6 by 1 initial condition; set using traj data
        # rpy_0E_initial, #initial rotation orientation; set using traj data

        csv_filepath,  #file path to traj filepath

        control_sample_period_s=1e-3
):
    #validate_np_array(arr=p_initial, arr_name="p_initial", correct_shape=(6,))
    #validate_np_array(arr=p_final, arr_name="p_final", correct_shape=(2,))
    # builder = DiagramBuilder()
    arm = models[0]
```

```
# print(plant.GetModelInstanceName(arm[0]))

# Read CSV file with trajectory and set sim duration
traj_times, od, odd, oddd = get_input_trajectory(csv_filepath)
simulation_duration_s = float(traj_times[-1])

id_output_truncator: TruncateVector = builder.AddNamedSystem(
    "truncate",
    TruncateVector()
)
trajectory: PrePlannedTrajectoryMold = builder.AddNamedSystem(
    "trajectory",
    PrePlannedTrajectoryMold(
        name="Desired Trajectory",
        times=traj_times,
        od=od,
        odd=odd,
        oddd=oddd
    )
)

# contact_mapper: WrenchOutput = builder.AddNamedSystem(
#     "contact_mapper",
#     WrenchOutput()
# )

controller: OperationalImpedanceController = builder.AddNamedSystem(
    "controller",
    OperationalImpedanceController(
        M_d=M_d,
        K_P=K_P,
        K_D=K_D,
        plant=plant,
        R_0D=R_0D,  # assuming planar model
        w_0D=w_0D  # assuming planar model
    ),
)

direct_kins: forwardkin = builder.AddNamedSystem(
    "forwardkins",
    forwardkin(
        plant=plant,
    )
)
# #Using the InbuiltInverse Dynamics Module: should already include the applied forces
# https://drake.mit.edu/doxygen_cxx/classdrake_1_1systems_1_1controllers_1_1_inverse_dynam
mode = InverseDynamics.kInverseDynamics
inversedynamics: InverseDynamics = builder.AddNamedSystem(
    "invdyn", InverseDynamics(plant,
                              mode=mode)
)

zoh: ZeroOrderHold = builder.AddNamedSystem(
    "sampled_oddot_e",
    ZeroOrderHold(
        period_sec=control_sample_period_s, vector_size=6
```

```
    ),
)

# High Level Control
# Impedance Control -> Inverse Dynamics Block -> Robot/Enviro ->Direct Kinematics ->Impeda
# This all needs to be updated
# ---------------------------------------------------------------------------------
builder.Connect(trajectory.get_output_port(),
                controller.get_input_port(2))

# Sending Desired Acceleration to Inverse Dynamics
builder.Connect(controller.get_output_port(0),
                inversedynamics.get_input_port_desired_acceleration())

# connecting output torques from inverse dynamics to the plant actuator
builder.Connect(
    inversedynamics.get_output_port(),
    id_output_truncator.get_input_port()
)
# connecting output torques from inverse dynamics to the plant actuator
builder.Connect(
    id_output_truncator.get_output_port(),
    zoh.get_input_port()
)
# Feedback of states to the controller
builder.Connect(
    plant.get_state_output_port(),
    controller.get_input_port(0)
)

builder.Connect(
    plant.get_state_output_port(),
    inversedynamics.get_input_port_estimated_state()
)

# Sending Force Feedback to Controller
# print("hello", plant.GetStateNames(arm[0]))
# print(plant.get_generalized_contact_forces_output_port(arm[0]).size())
builder.Connect(
    plant.get_generalized_contact_forces_output_port(arm[0]),
    controller.get_input_port(3)
)

# # Get vectorized contact force
# builder.Connect(
#     plant.get_contact_results_output_port(),
#     contact_mapper.get_input_port())
# builder.Connect(
#     contact_mapper.get_output_port(),
#     controller.get_input_port(3)
# )

# Sending State Feedback to direct kinematics calculation
builder.Connect(
    plant.get_state_output_port(),
    direct_kins.get_input_port()
```

```
    )

    # Connect direct kinematics output to controller
    builder.Connect(
        direct_kins.get_output_port(),
        controller.get_input_port(1)
    )

    # Add ZOH to avoid loop error
    builder.Connect(
        zoh.get_output_port(),
        plant.get_actuation_input_port()
    )
    # ==============================================================================

    # ------------------- Set up loggers ----------------------------------
    logger = dict()
    logger["force"] = LogVectorOutput(
        src=controller.get_output_port(3),
        builder=builder,
        publish_period=control_sample_period_s,
    )
    logger["force"].set_name("Force Logger")

    logger["op_space_state"] = LogVectorOutput(
        direct_kins.get_output_port(), builder, publish_period=1e-3
    )
    logger["x_tilde"] = LogVectorOutput(
        controller.get_output_port(1), builder, publish_period=1e-3
    )
    logger["x_tilde_dot"] = LogVectorOutput(
        controller.get_output_port(2), builder, publish_period=1e-3
    )

    # Build a `Diagram` object and use it to make a `Simulator` object for the diagram
    diagram: Diagram = builder.Build()
    diagram.set_name("Operational Impedance Control")
    simulator: Simulator = Simulator(diagram)
    root_context: Context = simulator.get_context()

    # Set initial conditions
    plant_context = plant.GetMyMutableContextFromRoot(root_context)
    p_0E_initial = od[:, 0]
    rpy_0E_initial = RotationMatrix(R_0D).ToRollPitchYaw().vector()
    q_initial = get_q_initial(plant, plant_context, p_0E_initial, rpy_0E_initial)
    plant.SetPositions(plant_context, arm[0], q_initial)
    plant.SetVelocities(plant_context, arm[0], np.zeros([6, ]))

    return diagram, simulator, root_context, simulation_duration_s, logger


def add_scene(discrete_update_period=1e-3):
    builder = DiagramBuilder()
    meshcat = Meshcat(port=MESHCAT_PORT)

    # Test for frames
```

```
urdf_path = (
    "generateMesh/ur5e/ur5e.urdf"
)
# Add models using a 'Parser' object (parses ".sdf" and ".urdf" files)
plant: MultibodyPlant
scene_graph: SceneGraph
plant: MultibodyPlant
scene_graph: SceneGraph
plant, scene_graph = AddMultibodyPlant(
    MultibodyPlantConfig(
        time_step=discrete_update_period,
        contact_model="hydroelastic_with_fallback",
        contact_surface_representation="triangle",
        discrete_contact_approximation="tamsi",
    ),
    builder,
)
parser = Parser(plant)
flag_user_JW = False
if flag_user_JW:
    arm = parser.AddModels(file_name="generateMesh/ur5e/ur5e.urdf")  # Returns ModelInstan
    roller = parser.AddModels(file_name="generateMesh/surface_sdf/free_cyl.sdf")
    # floor = parser.AddModels(file_name="generateMesh/surface_sdf/floor.sdf")
    mold = parser.AddModels(file_name="generateMesh/surface_sdf/mold_planar.sdf")
else:
    arm = parser.AddModels(
        file_name="/home/sjshah6/MAE263C/MAEC263C/generateMesh/ur5e/ur5e.urdf")  # Returns
    roller = parser.AddModels(file_name="/generateMesh/surface_sdf/free_cyl.sdf")
    #floor = parser.AddModels(file_name="generateMesh_Shirley/surface_sdf/floor.sdf")
    mold = parser.AddModels(file_name="generateMesh_Shirley/surface_sdf/mold_planar.sdf")
# Weld the rigid box to the world so that it's fixed during simulation.
# The top surface passes the world's origin.
# plant.WeldFrames(plant.world_frame(),
#                  plant.GetFrameByName("floor_surface"))
plant.WeldFrames(plant.world_frame(),
                 plant.GetFrameByName("mold_to_base"))
roller_joint = RevoluteJoint(
    name="roller_joint",
    frame_on_parent=plant.GetFrameByName("roller_attachment_parent"),
    frame_on_child=plant.GetFrameByName("roller_attachment_frame"),
    axis=np.array([0, 1, 0]),
    damping=0.001)
plant.AddJoint(roller_joint)

# Visualize frames
AddFrameTriadIllustration(
    scene_graph=scene_graph,
    frame=plant.GetFrameByName("roller_attachment_parent"))

# Create a 'MeshcatVisualizer' to view our scene graph using Meshcat and add it to our dia
visualizer: MeshcatVisualizer = MeshcatVisualizer.AddToBuilder(
    builder, scene_graph, meshcat
)

plant.Finalize()
# models = (arm, roller, floor, mold)
```

```python
    models = (arm, roller, mold)

    return builder, plant, scene_graph, visualizer, models


if __name__ == "__main__":
    print('start')
    trial_num = 4

    # Need to Replace following code with definitions
    np.set_printoptions(suppress=True, precision=5, floatmode="fixed")
    # meshcat = Meshcat(port=MESHCAT_PORT)
    # target_real_time_rate = 1
    # discrete_update_period = 1e-3
    # simulation_duration = 2
    faulthandler.enable()
    builder, plant, scene_graph, visualizer, models = add_scene()

    # Set visualize_frames to True to see that body the frames
    visualize_frames = False
    if visualize_frames:
        triad_radius = 0.01
        triad_length = 0.025
        triad_opacity = 1
        vis_frame(scene_graph, plant, triad_length, triad_radius, triad_opacity)

    print(plant.GetModelInstanceName(models[1][0]))

    control_sample_period_s = 1e-3
    roller = models[1]

    # --------------------- Build diagram and prepare for sim
    theta = np.pi / 4
    test_case = "Planar"  # or "Push" or "Planar"
    if test_case == "Run" or test_case == "Push":
        print("Running Push Test")
        M_d = np.diag([100.0, 100.0, 2000000.0, 10000, 1000, 10000])
        K_P = np.diag([1000000.0, 100000.0, 10000000000.0, 10000000, 10000000, 1000000000])
        K_Dx = 2 * np.sqrt(M_d[0][0] * K_P[0][0])
        K_Dy = 2 * np.sqrt(M_d[1][1] * K_P[1][1])
        K_Dz = 2 * np.sqrt(M_d[2][2] * (1e6 + K_P[2][2]))
        K_Dphi = 2 * np.sqrt(M_d[3][3] * (1e6 + K_P[3][3]))
        K_Dtheta = 2 * np.sqrt(M_d[4][4] * (1e6 + K_P[4][4]))
        K_Dpsi = 2 * np.sqrt(M_d[5][5] * (1e6 + K_P[5][5]))
        K_D = np.asarray(np.diag([K_Dx, K_Dy, K_Dz, K_Dphi, K_Dtheta, K_Dpsi]))
    elif test_case == "Planar":
        print("Running Planar Test")

        # Case 1
        # M_d = np.diag([100.0, 100.0, 2000000.0, 10000, 1000, 10000])
        # K_P = np.diag([1000000.0, 100000.0, 10000000000.0, 10000000, 10000000, 1000000000])

        # Case 2
        # M_d = np.diag([1000.0, 200.0, 3000000.0, 20000, 2000, 20000])
        # K_P = np.diag([2000000.0, 200000.0, 20000000000.0, 20000000, 20000000, 2000000000])
```

```
    # Case 3
    M_d = np.diag([20.0, 50.0, 200.0, 100, 100, 1000])
    K_P = np.diag([1000.0, 100.0, 1000.0, 1000, 1000, 10000])


    #M_d = np.diag([1, 1, 1, 1, 1, 1])
    #K_P = np.diag([1, 1, 1, 1, 1, 1])
    #M_d = np.diag([100, 100, 100, 50, 50, 50])
    #K_P = np.diag([2, 2, 1, 1, 1, 1])
    K_Dx = 2 * np.sqrt(M_d[0][0] * K_P[0][0])
    K_Dy = 2 * np.sqrt(M_d[1][1] * K_P[1][1])
    K_Dz = 2 * np.sqrt(M_d[2][2] * (1e6 + K_P[2][2]))
    K_Dphi = 2 * np.sqrt(M_d[3][3] * (1e6 + K_P[3][3]))
    K_Dtheta = 2 * np.sqrt(M_d[4][4] * (1e6 + K_P[4][4]))
    K_Dpsi = 2 * np.sqrt(M_d[5][5] * (1e6 + K_P[5][5]))
    K_D = np.asarray(np.diag([K_Dx, K_Dy, K_Dz, K_Dphi, K_Dtheta, K_Dpsi]))
    print(K_D)

diagram, simulator, root_context, sim_duration, logger = buildDiagram(
    plant=plant,
    builder=builder,
    models=models,
    M_d=M_d,
    K_P=K_P,
    K_D=K_D,
    R_0D=np.array([[np.cos(theta), -np.sin(theta), 0],
                   [np.sin(theta), np.cos(theta), 0],
                   [0, 0, 1]]),
    w_0D=np.zeros(3),
    #p_0E_initial=p_0E_initial,  # 6 by 1 initial condition
    #rpy_0E_initial=rpy_0E_initial,  # initial rotation orientation
    csv_filepath="generateTrajectory/trajectoryData/pathPlanar45.csv"  # file path to traj
)
simulator.set_target_realtime_rate(1.0)

# Start listening for events in Meshcat, and start the sim
visualizer.StartRecording()
simulator.AdvanceTo(sim_duration)

# Stop listening for events in Meshcat
visualizer.StopRecording()
visualizer.PublishRecording()

print("Done with Simulation:Plotting Results........")
# Plot force
force = logger["force"].FindLog(simulator.get_context())
t = force.sample_times()
f = force.data()[:6, :]
fig = plt.figure(figsize=(15, 10))
axes = [[], []]
title_pos = ["X", "Y", "Z"]
title_rpy = ["Roll", "Pitch", "Yaw"]
for j, variable in enumerate(title_pos):
    axes[0].append(fig.add_subplot(2, 3, 0 * 3 + j + 1))
    axes[0][j].set_title(f"End Effector " + variable + " Force")
    axes[0][j].set_xlabel(f"Time [s]")
```

```
    axes[0][j].set_ylabel(f"Force [N]")
    axes[0][j].plot(t, f[j, :], color="#CE2E35")
for j, variable in enumerate(title_pos):
    axes[1].append(fig.add_subplot(2, 3, 3 + j + 1))
    axes[1][j].set_title(f"End Effector " + variable + " Torque")
    axes[1][j].set_xlabel(f"Time [s]")
    axes[1][j].set_ylabel(f"Torque [N*m]")
    axes[1][j].plot(t, f[3 + j, :], color="#CE2E35")

# Plot errors
op_space_state = logger["op_space_state"].FindLog(simulator.get_context())
t = op_space_state.sample_times()
x = op_space_state.data()[:6, :]
fig = plt.figure(figsize=(15, 10))
axes = [[], []]
title_pos = ["X", "Y", "Z"]
title_rpy = ["Roll", "Pitch", "Yaw"]
for j, variable in enumerate(title_pos):
    axes[0].append(fig.add_subplot(2, 3, 0 * 3 + j + 1))
    axes[0][j].set_title(f"End Effector " + variable + " Position")
    axes[0][j].set_xlabel(f"Time [s]")
    axes[0][j].set_ylabel(f"Position [m]")
    axes[0][j].plot(t, x[j, :], color="#CE2E35")
for j, variable in enumerate(title_rpy):
    axes[1].append(fig.add_subplot(2, 3, 3 + j + 1))
    axes[1][j].set_title(f"End Effector " + variable)
    axes[1][j].set_xlabel(f"Time [s]")
    axes[1][j].set_ylabel(f"Angle [deg]")
    axes[1][j].plot(t, np.rad2deg(x[3 + j, :]), color="#CE2E35")

# End Effector Generalized Velocity
x_dot = op_space_state.data()[6:, :]
fig = plt.figure(figsize=(15, 10))
axes = [[], []]
title_vel = ["Vx", "Vy", "Vz"]
title_omega = ["Wx", "Wy", "Wz"]
for j, variable in enumerate(title_pos):
    axes[0].append(fig.add_subplot(2, 3, 0 * 3 + j + 1))
    axes[0][j].set_title(f"End Effector " + variable)
    axes[0][j].set_xlabel(f"Time [s]")
    axes[0][j].set_ylabel(f"Velocity [m/s]")
    axes[0][j].plot(t, x_dot[j, :], color="#CE2E35")
for j, variable in enumerate(title_omega):
    axes[1].append(fig.add_subplot(2, 3, 3 + j + 1))
    axes[1][j].set_title(f"End Effector " + variable)
    axes[1][j].set_xlabel(f"Time [s]")
    axes[1][j].set_ylabel(f"Angular Velocity [deg/s]")
    axes[1][j].plot(t, np.rad2deg(x_dot[3 + j, :]), color="#CE2E35")

spatial_error_log = logger["x_tilde"].FindLog(simulator.get_context())
t = spatial_error_log.sample_times()
x_e = spatial_error_log.data()
fig = plt.figure(figsize=(15, 10))
axes = [[], []]
for j, variable in enumerate(title_pos):
    axes[0].append(fig.add_subplot(2, 3, 0 * 3 + j + 1))
```

```
    axes[0][j].set_title(f"End Effector " + variable + " Error")
    axes[0][j].set_xlabel(f"Time [s]")
    axes[0][j].set_ylabel(f"Error [m]")
    axes[0][j].plot(t, x_e[j, :], color="#CE2E35")
    axes[0][j].axhline(
        y=0,
        linestyle="--",
        color="k",
    )
for j, variable in enumerate(title_rpy):
    axes[1].append(fig.add_subplot(2, 3, 3 + j + 1))
    axes[1][j].set_title(f"End Effector " + variable + " Error")
    axes[1][j].set_xlabel(f"Time [s]")
    axes[1][j].set_ylabel(f"Error [deg]")
    axes[1][j].plot(t, np.rad2deg(x_e[3 + j, :]), color="#CE2E35")
    axes[1][j].axhline(
        y=0,
        linestyle="--",
        color="k",
    )

pos_data = pd.DataFrame({"X[m]": x[0, :], "Y[m]": x[1, :], "Z[m]": x[2,:],
                         "Roll[deg]": x[3, :], "Pitch[deg]": x[4, :], "Yaw[deg]": x[5,: ]}
pos_data_filepath = "Trial" + str(trial_num) + "_PosData.csv"
pos_data.to_csv(pos_data_filepath)

vel_data = pd.DataFrame({"Time":t,"X_dot[m/s]": x_dot[0, :], "Y_dot[m/s]": x_dot[1, :], "Z_
                         "Roll_dot[deg/s]": x_dot[3, :], "Pitch_dot[deg/s]": x_dot[4, :],
vel_data_filepath = "Trial" + str(trial_num) + "_velData.csv"
vel_data.to_csv(vel_data_filepath)



error_data = pd.DataFrame({"Time":t,"X_E[m]": x_e[0, :], "Y_E[m]": x_e[1, :], "Z_E[m]": x_e
                           "Roll_E[deg]": x_e[3, :], "Pitch_E[deg]": x_e[4, :], "Yaw_E[deg
error_data_filepath = "Trial" + str(trial_num) + "_ErrorData.csv"
error_data.to_csv(error_data_filepath)

force_data = pd.DataFrame({"Time":t, "F_X[N]": f[0, :], "F_Y[N]": f[1, :], "F_Z[N]": f[2,
                           "F_Roll[Nm]": f[3, :], "F_Pitch[Nm]": f[4, :], "F_Yaw[Nm]": f[5
force_data_filepath = "Trial" + str(trial_num) + "_ForceData.csv"
force_data.to_csv(force_data_filepath)

plt.show()
```

XI. Appendix: Assorted MATLAB Code

```
% ==============================================================================
%% MAE C263C Project Path Generation
% Description: defines a mold surface, then generates trajectories for each
% pass to cover the surface.
clear variables; clc; close all;

%% End Effector Parameters
maxOff = 2.5; % [mm] Max allowable vertical offset from surface
N_tow = 30; % [] Max number of tows
w_tow = 3.175; % [mm] Width of each tow
d_max = N_tow * w_tow; % [mm] Max offset distance from reference path

%% Surface Definition and Parameterization
mold_type = 'Planar';
theta = 45; % [deg] Laying angle wrt mold orientation
f = [1; 0; 0]; % Fixed mold orientation
l = rotz(theta)*f;
M = [250; 300; 400];
fileout = ['trajectoryData/path', mold_type, num2str(theta),'.mat'];

% Surface Parameterization
N = 101;
umin = 0; umax = 1;
vmin = 0; vmax = 1;
uvLim = [umin, umax, vmin, vmax];
u = linspace(umin,umax,N);
v = linspace(vmin,vmax,N);

figure
%fsurf(@getBezierPtX,@getBezierPtY,@getBezierPtZ,uvLim)
%grid on
hold on

%% Initial Path Generation
% Initial path reference plane
p = [0; 0; -1]; % Projection vector
abc = reshape(cross(p,l), [1,3]);
d = -M'*abc';
abcd = [abc, d];
nInt = 100;

[uInit,vInit] = getSurfacePlaneIntersect(abcd,uvLim,nInt);

% Test plot
for i = 1:nInt
    pInit = getBezierPt(uInit(i),vInit(i));
    plot3(pInit(1),pInit(2),pInit(3),'rx')
end

%% Offset Path Generation
% Take in reference path, output offset paths

% "Forward" offset direction
pathCountForward = 0;
```

```
uRef = uInit;
vRef = vInit;
offsetDir = 1;
flag_offset = true;

while flag_offset
    [uOff,vOff,nLostPts] = getOffsetPath(uRef,vRef,offsetDir,uvLim,nInt,w_tow,d_max);
    if isempty(uOff)
        flag_offset = false; % redundant?
        break;
    end

    % Least squares regression
    P_data = zeros(length(uOff),3);
    for i = 1:length(uOff)
        P_data(i,:) = getBezierPt(uOff(i),vOff(i))';
    end
    fitPlane = (P_data'*P_data) \ P_data' * -ones(length(uOff),1);
    abcdFitPlane = [fitPlane', 1];

    % Estimate desired number of points based on initial path spacing
    l_init = vecnorm(getBezierPt(uInit(1),vInit(1))...
                   - getBezierPt(uInit(end),vInit(end)));
    l_off = vecnorm(P_data(1,:)-P_data(end,:));
    nOff = ceil(nInt*l_off/l_init);

    % Generate new offset points
    p_ref = mean(P_data,1)'; % average of offset traj points for reference
    [uOff, vOff] = getSurfacePlaneIntersect(abcdFitPlane,uvLim,nOff,p_ref);

    % Store path
    pathCountForward = pathCountForward+1;
    uForward{pathCountForward} = uOff;
    vForward{pathCountForward} = vOff;
    uRef = uOff;
    vRef = vOff;

    % Test plot
    plot3(P_data(:,1),P_data(:,2),P_data(:,3),'rx')

end

% "Backward" offset direction
pathCountBackward = 0;
uRef = uInit;
vRef = vInit;
offsetDir = -1;
flag_offset = true;

while flag_offset
    % Generate offset points
    [uOff,vOff,nLostPts] = getOffsetPath(uRef,vRef,offsetDir,uvLim,nInt,w_tow,d_max);
    if isempty(uOff)
        flag_offset = false; % redundant?
        break;
    end
```

```
    % Least squares regression
    P_data = zeros(length(uOff),3);
    for i = 1:length(uOff)
        P_data(i,:) = getBezierPt(uOff(i),vOff(i))';
    end
    fitPlane = (P_data'*P_data) \ P_data' * -ones(length(uOff),1);
    abcdFitPlane = [fitPlane', 1];

    % Estimate desired number of points based on initial path spacing
    l_init = vecnorm(getBezierPt(uInit(1),vInit(1))...
                   - getBezierPt(uInit(end),vInit(end)));
    l_off = vecnorm(P_data(1,:)-P_data(end,:));
    nOff = ceil(nInt*l_off/l_init);

    % Generate new offset points
    p_ref = mean(P_data,1)'; % average of offset traj points for reference
    [uOff, vOff] = getSurfacePlaneIntersect(abcdFitPlane,uvLim,nOff,p_ref);

    % Store path
    pathCountBackward = pathCountBackward+1;
    uBackward{pathCountBackward} = uOff;
    vBackward{pathCountBackward} = vOff;
    uRef = uOff;
    vRef = vOff;

    % Test plot
    for i = 1:length(uOff)
        pOff = getBezierPt(uOff(i),vOff(i));
        plot3(pOff(1),pOff(2),pOff(3),'rx')
    end
end


% Complete trajectory cell array
uComplete = cell(pathCountBackward + 1 + pathCountForward, 1);
vComplete = cell(pathCountBackward + 1 + pathCountForward, 1);

for i = 1:pathCountBackward
    uComplete{i} = uBackward{pathCountBackward-i+1}; % Start from last offset
    vComplete{i} = vBackward{pathCountBackward-i+1};
end

i_init = pathCountBackward+1; % Path index for initial reference path
uComplete{i_init} = uInit;
vComplete{i_init} = vInit;

for i = 1:pathCountForward
    i_forward = i_init + i;
    uComplete{i_forward} = uForward{i}; % Start from last offset
    vComplete{i_forward} = vForward{i};
end

grid on
hold off
```

```
save(fileout, uComplete, vComplete)

% ==============================================================================
%% MAE C263C Project Trajectory Generation
% Description: from a given roller path, generate arm end effector
% trajectory.
%
% Output: .csv file with time, position, velocity, and acceleration wrt
% base
clear variables; clc; close all;

traj_data = 'trajectoryData/pathPlanar45';
enum_test = 'Planar'; % 'Planar', 'Push', or 'Run'
n_run = 5;

switch enum_test
    case 'Push'
        uComplete{1} = [0.5, 0.51]; vComplete{1} = [0.5, 0.51];
        v_case = 10;

    case 'Planar'
        file_in = [traj_data, '.mat'];
        load(file_in); % uComplete, vComplete
        v_case = 200;

        enum_test = '';

    case 'Run'
        file_in = [traj_data, '.mat'];
        load(file_in); % uComplete, vComplete

        uRun = uComplete{n_run}; vRun = vComplete{n_run};
        uComplete = {}; vComplete = {};
        uComplete{1} = uRun; vComplete{1} = vRun;

        v_case = 200;
        enum_test = [enum_test, num2str(n_run)];
end

file_out = [traj_data, enum_test, '.csv'];

control_sample_period = 1e-3;

%% End Effector Parameters
N_tow = 30; % [] Max number of tows
w_tow = 3.175; % [mm] Width of each tow
w_roller = N_tow*w_tow; % [mm] roller width (see free_cyl.sdf)
r_roller = 20; % [mm] roller radius (from free_cyl.sdf)

% Get vector pointing from end effector edge to tool frame origin,
% expressed in base frame
p_0_EC = [-w_tow; 0; r_roller];
p_0_home = [600; 0; 100]; % [mm] Home position wrt base
p_0_0M = [p_0_home(1)-250; -300; 0]; % [mm] Mold origin wrt base
p_vert_offset = [0; 0; -2]; % [mm] Target distance under mold surface
p_helper_offset = -p_vert_offset;
```

```
p_total_offset = p_0_EC + p_0_0M + p_vert_offset;

% Set up velocities
v_roll_des = v_case; % [mm/s] Tool velocity while rolling
v_down_des = v_case; % [mm/s] Tool velocity while moving down towards mold
v_insert_des = 10; % [mm/s] Tool velocity while moving into mold from surface
v_free_des = 500; % [mm/s] Tool velocity in free space
vert_offset = [0; 0; 50]; % [mm] Tool should lift off mold between runs

% ---------------------------------------------------------------------------------------------

%% Trajectory Generation
n_runs = length(uComplete);
t = cell(n_runs,1); x = cell(n_runs,1);
dt = control_sample_period;
for i = 1:n_runs
    % Define waypoints
    if length(uComplete{i}) == 2
        p0 = getBezierPt(mean(uComplete{i}), mean(vComplete{i})) + p_total_offset;
    elseif length(uComplete{i}) < 2
        error('Less than 2 points in input trajectory run!')
    else
        p0 = getBezierPt(uComplete{i}(floor(end/2)), vComplete{i}(floor(end/2))) + p_total_off
    end
    pf1 = getBezierPt(uComplete{i}(1), vComplete{i}(1)) + p_total_offset;
    pf2 = getBezierPt(uComplete{i}(end), vComplete{i}(end)) + p_total_offset;
    p0_above = p0 + vert_offset;
    pf1_above = pf1 + vert_offset;
    pf2_above = pf2 + vert_offset;
    waypts = cell(10,1);
    waypts{1} = [p0_above, p0 + p_helper_offset];
    waypts{2} = [p0 + p_helper_offset, p0];
    waypts{3} = [p0, pf1];
    waypts{4} = [pf1, pf1_above];
    waypts{5} = [pf1_above, p0_above];
    waypts{6} = [p0_above, p0 + p_helper_offset];
    waypts{7} = [p0 + p_helper_offset, p0];
    waypts{8} = [p0, pf2];
    waypts{9} = [pf2, pf2_above];
    if i ~= n_runs % Add first above pt of next run
        if length(uComplete{i+1}) == 2
            p0_next = getBezierPt(mean(uComplete{i+1}), mean(vComplete{i+1})) + p_total_offset
        elseif length(uComplete{i+1}) < 2
            error('Less than 2 points in input trajectory run!')
        else
            p0_next = getBezierPt(uComplete{i+1}(floor(end/2)), vComplete{i+1}(floor(end/2)))
        end
        p0_above_next = p0_next + vert_offset;
    else
        p0_above_next = p_0_home;
    end
    waypts{10} = [pf2_above, p0_above_next];

    % Define desired velocities
    v = cell(10,1);
    v{1} = v_down_des;
```

```
    v{2} = v_insert_des;
    v{3} = v_roll_des;
    v{4} = v_free_des;
    v{5} = v_free_des;
    v{6} = v_down_des;
    v{7} = v_insert_des;
    v{8} = v_roll_des;
    v{9} = v_free_des;
    v{10} = v_free_des;

    % Get trapezoidal velocity trajectory
    p = cell(10,1); pd = cell(10,1); pdd = cell(10,1); t_samples = cell(10,1);
    for i_p = 1:10
        n_pts{i_p} = ceil(vecnorm(waypts{i_p}(:,2) - waypts{i_p}(:,1)) / v{i_p} / dt);
        [p{i_p}, pd{i_p}, pdd{i_p}, t_samples{i_p}] = ...
            trapveltraj(waypts{i_p}, n_pts{i_p}, 'PeakVelocity', v{i_p});

        % Determine end time (trapveltraj may not take up all of t_samples)
        traj = [t_samples{i_p}; p{i_p}; pd{i_p}; pdd{i_p}];
        traj = unique(traj.','rows','stable').';
        t_samples{i_p} = traj(1,:);
        p{i_p} = traj(2:4,:);
        pd{i_p} = traj(5:7,:);
        pdd{i_p} = traj(8:10,:);
    end

    % Interpolate to controller/drake sim time
    t0 = 0;
    t_interp = cell(10,1);
    x_interp = cell(10,1);
    for i_p = 1:10
        tf = floor(t_samples{i_p}(end)/dt)*dt + dt;
        t_interp{i_p} = linspace(t0, tf, tf/dt + 1);
        x_interp{i_p} = interp1(t_samples{i_p}, [p{i_p}; pd{i_p}; pdd{i_p}]', t_interp{i_p}, '
    end

    % Mash phases for this run together
    t_run = t_interp{1}; x_run = x_interp{1};
    for i_p = 2:10
        t_run = [t_run, t_interp{i_p} + t_run(end) + dt]; % Don't want repeat times
        x_run = [x_run, x_interp{i_p}];
    end

    t{i} = t_run;
    x{i} = x_run;
end

% Accumulate trajectory
t_out = t{1};
x_out = x{1};
for i = 2:n_runs
    t_out = [t_out, t{i} + t_out(end) + dt];
    x_out = [x_out, x{i}];
end

% Convert mm to meters
```

```
x_out = x_out./1000;

% Rearrange and write to .csv file
header = ["Times", "odx", "ody", "odz", "oddx", "oddy", "oddz", "odddx", "odddy", "odddz"];
data = [t_out', x_out'];
writematrix(header, file_out);
writematrix(data, file_out, 'WriteMode', 'append');

%% Plotting
plot3(x_out(1,:), x_out(2,:), x_out(3,:), 'k.')
hold on
plot3(x_out(1,1), x_out(2,1), x_out(3,1), 'ro','MarkerSize',10,'LineWidth',3)
plot3(x_out(1,end), x_out(2,end), x_out(3,end), 'rx','MarkerSize',10,'LineWidth',3)
plot3([-0.25, -0.25, 0.25, 0.25]+p_0_home(1)/1000,...
      [-0.3 0.3 -0.3 0.3], [0 0 0 0],'r.', 'MarkerSize',15)
plot3(0,0,0, 'ko', 'LineWidth',3, 'MarkerSize',15)
title('Planar Roller Trajectory in World Frame')
xlabel('X [m]')
ylabel('Y [m]')
zlabel('Z [m]')
grid on

% ------------------------------------------------------------------------------------------
function [p] = getBezierPt(u,v)
    % Control Points
    P = cell(3);
    P{1,1} = [0   0   0]'; P{1,2} = [250    0 150]'; P{1,3} = [500  100 100]';
    P{2,1} = [0 250  50]'; P{2,2} = [300  300  50]'; P{2,3} = [450  200   0]';
    P{3,1} = [0 500   0]'; P{3,2} = [200  500 100]'; P{3,3} = [450  600  50]';
    %
    % P{1,1} = [0   0   50]'; P{1,2} = [250    0   50]'; P{1,3} = [500   0   50]';
    % P{2,1} = [0 300   50]'; P{2,2} = [250  300   50]'; P{2,3} = [500  300  50]';
    % P{3,1} = [0 600   50]'; P{3,2} = [250  600   50]'; P{3,3} = [500  600  50]';

    % Surface Point Calculation
    S_uv = [0; 0; 0];
    for i = 0:2
        for j = 0:2
            B_i2 = nchoosek(2,i) * u^i * (1-u)^(2-i);
            B_j2 = nchoosek(2,j) * v^j * (1-v)^(2-j);
            S_uv = S_uv + B_i2*B_j2*P{i+1,j+1};
        end
    end

    p = S_uv;
end

% ==========================================================================================
% MAE C263C Project: generateSurface (mesh generation)
clear variables; clc; close all;

% Note: for getBezierPt, incrementing u corresponds mostly to y-motion,
% while incrementing v corresponds mostly to x-motion.

% Define u,v grid
N_uv = 51;
```

```
umin = 0; umax = 1;
vmin = 0; vmax = 1;
u = linspace(umin, umax, N_uv);
u = [u(1), mean(u(1:2)), u(2:end-1), mean(u(end-1:end)), u(end)];
v = linspace(umin, umax, N_uv);
v = [v(1), mean(v(1:2)), v(2:end-1), mean(v(end-1:end)), v(end)];
N_uv = N_uv+2;

% Get Bezier pts (mesh top)
offset = [0, 0, 100];
P_top = cell(N_uv);
for i = 1:N_uv
    for j = 1:N_uv
        P_top{i,j} = getBezierPt(u(j), v(i))' + offset;
    end
end

% Bottom of mesh
extractXY = [eye(2), zeros(2,1); zeros(1,3)];
P_bot = cell(N_uv);
for i = 1:N_uv
    for j = 1:N_uv
        P_bot{i,j} = P_top{i,j} * extractXY;
    end
end

% Estimate desired mesh spacing for walls
delta = vecnorm(P_bot{1,2}-P_bot{1,1})*0.7;

% Bottom ("south") wall
P_south = cell(N_uv);
for i = 1:N_uv % [umax:umin, vmin] -> P_west{1,1} ~ getBezierPt(umax,vmin)
    d_bot2top = vecnorm(P_top{i,1}-P_bot{i,1});
    delta_i = d_bot2top / ceil(d_bot2top/delta);
    for j = 1:N_uv % bottom to top
        p = P_bot{i,1} + delta_i*(j-1)*unit(P_top{i,1}-P_bot{i,1});
        if p(3) > P_top{i,1}(3)
            P_south{i,j} = [NaN NaN NaN];
        else
            P_south{i,j} = p;
        end
    end
end

% Left ("west") wall
P_west = cell(N_uv);
for i = N_uv:-1:1 % [umin, vmin:vmax]
    i_rev = N_uv-i+1;
    d_bot2top = vecnorm(P_top{1,i}-P_bot{1,i});
    delta_i = d_bot2top / ceil(d_bot2top/delta);
    for j = 1:N_uv % bottom to top
        p = P_bot{1,i} + delta_i*(j-1)*unit(P_top{1,i}-P_bot{1,i});
        if p(3) > P_top{1,i}(3)
            P_west{i_rev,j} = [NaN NaN NaN];
        else
            P_west{i_rev,j} = p;
```

```
        end
    end
end

% Right ("east") wall
P_east = cell(N_uv);
for i = 1:N_uv % [umax, vmin:vmax]
    d_bot2top = vecnorm(P_top{end,i}-P_bot{end,i});
    delta_i = d_bot2top / ceil(d_bot2top/delta);
    for j = 1:N_uv % bottom to top
        p = P_bot{end,i} +  delta_i*(j-1)*unit(P_top{end,i}-P_bot{end,i});
        if p(3) > P_top{end,i}(3)
            P_east{i,j} = [NaN NaN NaN];
        else
            P_east{i,j} = p;
        end
    end
end

% Top ("north") wall
P_north = cell(N_uv);
for i = 1:N_uv % [umin:umax, vmax]
    i_rev = N_uv-i+1;
    d_bot2top = vecnorm(P_top{i,end}-P_bot{i,end});
    delta_i = d_bot2top / ceil(d_bot2top/delta);
    for j = 1:N_uv % bottom to top
        p = P_bot{i,end} +  delta_i*(j-1)*unit(P_top{i,end}-P_bot{i,end});
        if p(3) > P_top{i,end}(3)
            P_north{i_rev,j} = [NaN NaN NaN];
        else
            P_north{i_rev,j} = p;
        end
    end
end

% Generate surface normals
N_top = generateSurfaceNormals(P_top);
N_bot = generateSurfaceNormals(P_bot);
N_west = generateSurfaceNormals(P_west);
N_south = generateSurfaceNormals(P_south);
N_north = generateSurfaceNormals(P_north);
N_east = generateSurfaceNormals(P_east);
% Note: N_bot has normals in the wrong direction due to the definition of
% P_bot. This is quick-fixed later by saving the values as their negative.

% Extract points
PN_top_arr = zeros(N_uv^2,6);
PN_bot_arr = zeros(N_uv^2,6);
PN_west_arr = zeros(N_uv^2,6);
PN_south_arr = zeros(N_uv^2,6);
PN_north_arr = zeros(N_uv^2,6);
PN_east_arr = zeros(N_uv^2,6);
for i = 1:N_uv
    for j = 1:N_uv
        PN_top_arr(N_uv*(i-1)+j, :) = [P_top{i,j}, N_top{i,j}];
        PN_bot_arr(N_uv*(i-1)+j, :) = [P_bot{i,j}, -N_bot{i,j}]; % N_bot fix
```

```
        PN_west_arr(N_uv*(i-1)+j, :) = [P_west{i,j}, N_west{i,j}];
        PN_south_arr(N_uv*(i-1)+j, :) = [P_south{i,j}, N_south{i,j}];
        PN_north_arr(N_uv*(i-1)+j, :) =[P_north{i,j}, N_north{i,j}];
        PN_east_arr(N_uv*(i-1)+j, :) = [P_east{i,j}, N_east{i,j}];
    end
end

PN = [PN_top_arr; PN_bot_arr; PN_west_arr; PN_south_arr; PN_north_arr; PN_east_arr];
PN(any(isnan(PN),2),:) = [];
%P = unique(P,'rows');

% Test plot
figure
plot3(PN(:,1),PN(:,2),PN(:,3),'k.')

% Save data
% filename = ['../Surface Files/bezier3D_N', num2str(N_uv), '.txt'];
% writematrix(PN,filename,'Delimiter',' ');

function u = unit(v)
% Returns unit vector u pointing in same direction as v

    u = v./vecnorm(v);

end
```